



IIC2650: Fundamentos de Lenguajes de Computación

Tarea 1: Intro to Parsing

Miércoles 18-Marzo-2026

1 Objetivos

En estas primera semanas has conocido la base de las estructuras de los lenguajes de programación: la sintaxis. En esta tarea vas a construir el **parser** de una calculadora:

1. **Importante:** Cada ejercicio se debe hacer en **Python** y **Java**¹.
2. **Importante:** Se evaluarán los test entregados.
3. A partir de una sintaxis definida construirás el parse tree de un string válido.
4. Identificarás si un string no es válido.
5. En el caso de ambigüedad, entregarás todos los parse tree posibles.

Contenidos

1	Objetivos	1
2	Ejercicios	2
2.1	Parse Tree	2
2.2	Parse Error	2
2.3	Ambigüedad	3
3	Entrega, evaluación y classroom	3
4	Recomendaciones	4

¹Pueden instalarlos en python.org y en java.com

2 Ejercicios

Considere la siguiente gramática:

$$\begin{aligned} \langle expression \rangle &::= (\langle expression \rangle + \langle expression \rangle) \\ &| (\langle expression \rangle - \langle expression \rangle) \\ &| (\langle expression \rangle * \langle expression \rangle) \\ &| (\langle expression \rangle / \langle expression \rangle) \\ &| \langle number \rangle \end{aligned}$$

1. **Importante:** recuerda ejecutar los tests que te entregaremos para asegurar que tu programa funciona.
2. $\langle number \rangle$ es una secuencia de dígitos del 0 al 9.
3. Se te entregarán las clases de los nodos del árbol. Eres libre de modificarlos siempre y cuando no implique modificar los tests.

2.1 Parse Tree

Tu primer reto será obtener una representación en forma de árbol de esta. Defina la siguiente función:

```
parse(program_string) ->Ast:  
pass
```

Tal que `parse(program_string)` devuelve el árbol de derivación de `program_string`. Por ejemplo,

```
>>> Parser.parse("1")  
Constant(1)  
>>> Parser.parse("(9 + 0)")  
Sum(Constant(9), Constant(0))
```

2.2 Parse Error

Extiende tu implementación de `parse` para manejar expresiones inválidas:

```
>>> Parser.parse("(4)")  
Syntax error  
>>> Parser.parse(" 1 - 2 / 3")  
Syntax error
```

2.3 Ambigüedad

Considere la siguiente gramática levemente editada:

$$\begin{aligned} \langle expression \rangle &::= \langle expression \rangle + \langle expression \rangle \\ &| \langle expression \rangle - \langle expression \rangle \\ &| \langle expression \rangle * \langle expression \rangle \\ &| \langle expression \rangle / \langle expression \rangle \\ &| \langle number \rangle \end{aligned}$$

La cual es similar a la gramática anterior, pero sin paréntesis. Notablemente, es una gramática **ambigua**, lo que significa que un string puede tener más de un syntax tree válido. No definiremos orden de precedencia. Defina la función `trees` que retorna la cantidad de maneras distintas de parsear, o bien la cantidad de parse trees posibles. Por ejemplo:

```
>>> Parser.trees(" 5 + 6 * 7")
[Multiplication(Sum(Constant(5), Constant(6)), 7), Sum(Multiplication(Constant(5), Constant(6)), 7)]
```

Porque todos los posibles árboles de parsing son:



3 Entrega, evaluación y classroom

Para realizar la tarea, debe unirse al repositorio de GitHub Classroom del curso y aceptar la tarea 1.

- *Link classroom:* <https://classroom.github.com/a/Sya8lXkH>

El repositorio tiene la siguiente estructura de archivos para la tarea:

```
.gitignore
README.md
java/
python/
```

- Debe modificar el archivo `README.md` con su nombre y número de alumno. Si no los agrega, arriesgas que tu tarea no se pueda revisar.
- No se permite modificar los archivos de tests porque se utilizarán para evaluar la tarea. Si los modificas arriesgas una penalización.
- Es posible que nos comuniquemos con usted si hay algún trozo de código que genere dudas o sospechas. No responder o no poder explicar el código puede resultar en descuentos a su calificación.
- Sin embargo, siéntanse libres de modificar y agregar clases y métodos, siempre y cuando no modifique la interfaz de los tests.
- Se calificará el porcentaje de tests correctos: es decir que la nota de la tarea equivale a

$$\frac{\text{passing tests}}{\text{total tests}} * 6 + 1$$

- La **Fecha de Entrega:** es *Miércoles 01 de Abril* hasta las 23:59.
- No se permiten ni hay política de atraso.

4 Recomendaciones

- Los lenguajes de la tarea estan asociados a sus carpetas y archivos de tests correspondiente. Te recomendamos correr los tests para saber que pueden correr el código. Modifica los archivos base y ejecuta los tests para detectar cuales fallan. No está permitido modificar los tests.
- Los archivos tiene una estructura por defecto para las funciones que se te pide implementar, eres libre de extender las clases y funciones.
- Lee atentamente las notas resaltadas como **Importante** en el enunciado, ya que contiene restricciones sobre como puedes resolver el ejercicio.
- En caso de que falte información en el enunciado o los tests publicos para tomar decisiones respecto a la implementación, es libre de realizar suposiciones en como tratar con ciertos casos. Sin embargo, debe declarar en el archivo `README.md` que suposiciones realizo para cada parte.
- Si utilizas alguna herramienta como soporte para realizar su tarea, debes declararlo en el archivo `README.md`. Ten en cuenta las normas de integridad académica del curso.
- Su tarea será revisada manualmente por el cuerpo de ayudantes. Es por esto que se sugiere comentar su código y explicarlo de manera clara.